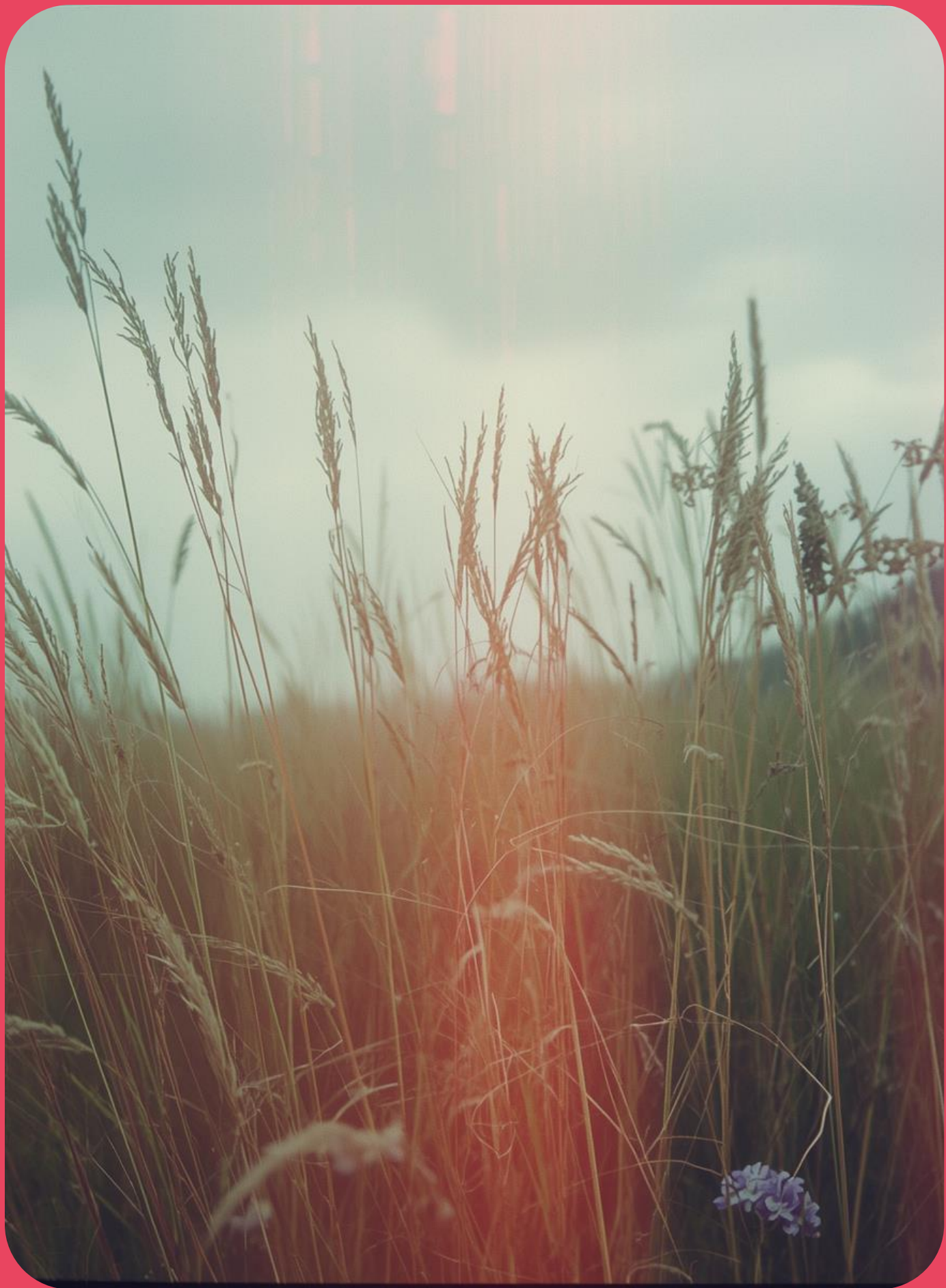


# Efecte de postprocesare

Cristian Lambru: 2025-26







# Cuprins

1. Introducere
2. Efectele de postprocesare *în spațiul de culoare*;
3. Procesul de netezire - „*blur*”;
4. Efectul de tip „*bloom*”;
5. Efectul de adâncime a câmpului vizual - „*depth of field*”.
6. Efectul de iluminare volumetrică - „*Volumetric Illumination*”.

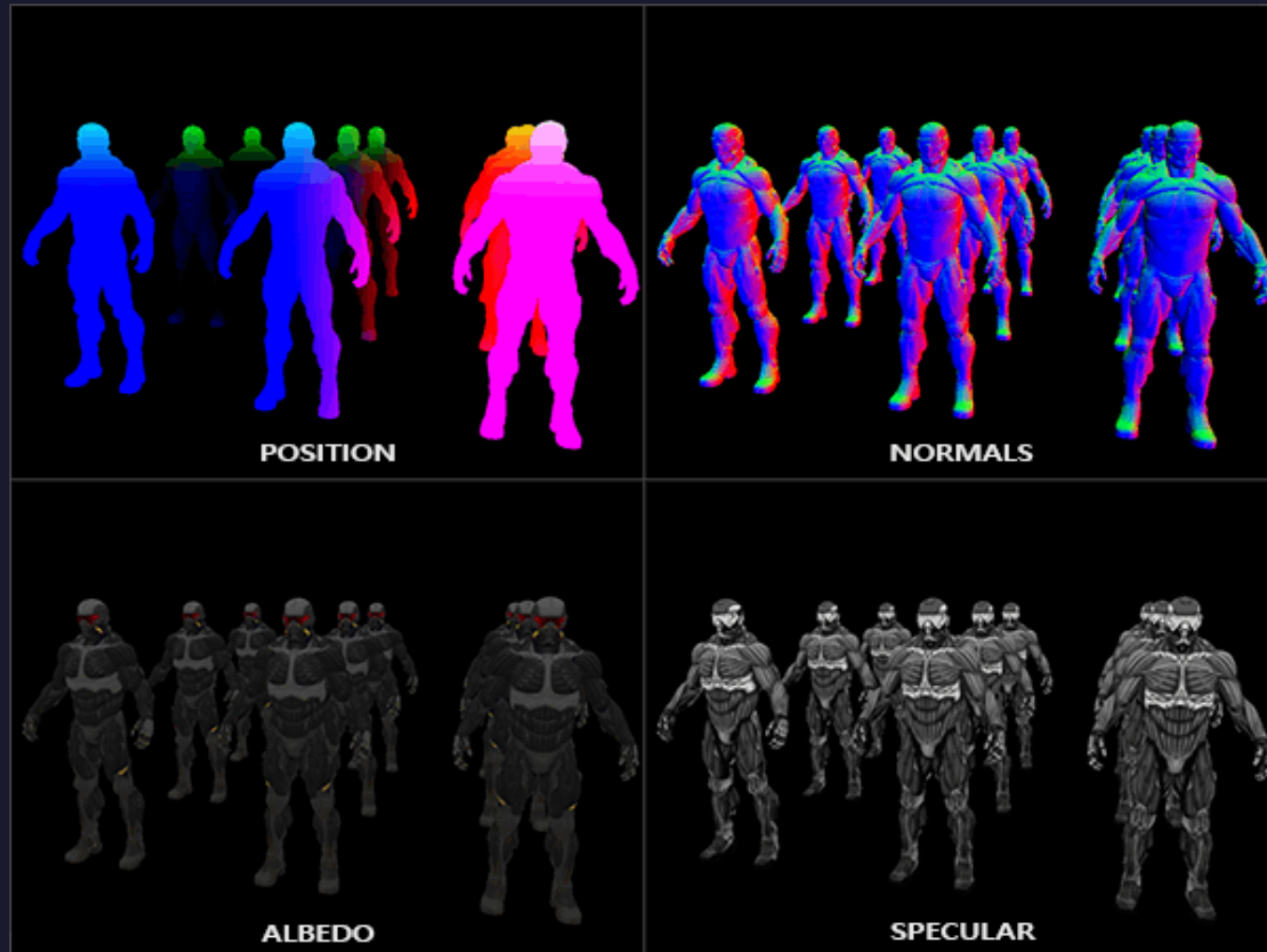


# Efectele de postprocesare (I)

Un **efect de postprocesare** este, în general, considerat un proces aplicat asupra culorilor pixelilor din imaginea rezultată **după desenarea scenei din perspectiva observatorului**, după ce **s-au realizat calculele de iluminare**.



# Efectele de postprocesare (II)



De cele mai multe ori, efectele de postprocesare au nevoie de **informații suplimentare la nivel de pixel**, în plus față de culoarea obținută după calculele de iluminare.

Astfel, unele efecte de postprocesare utilizează **informațiile geometrice**, calculate și stocate la nivel de pixel, similar cu cele obținute la primul **pas al metodei de desenare întârziată**, discutată săptămâna precedentă.

Recomandarea generală este să se păstreze informațiile geometrice în **spațiul de vizualizare**.





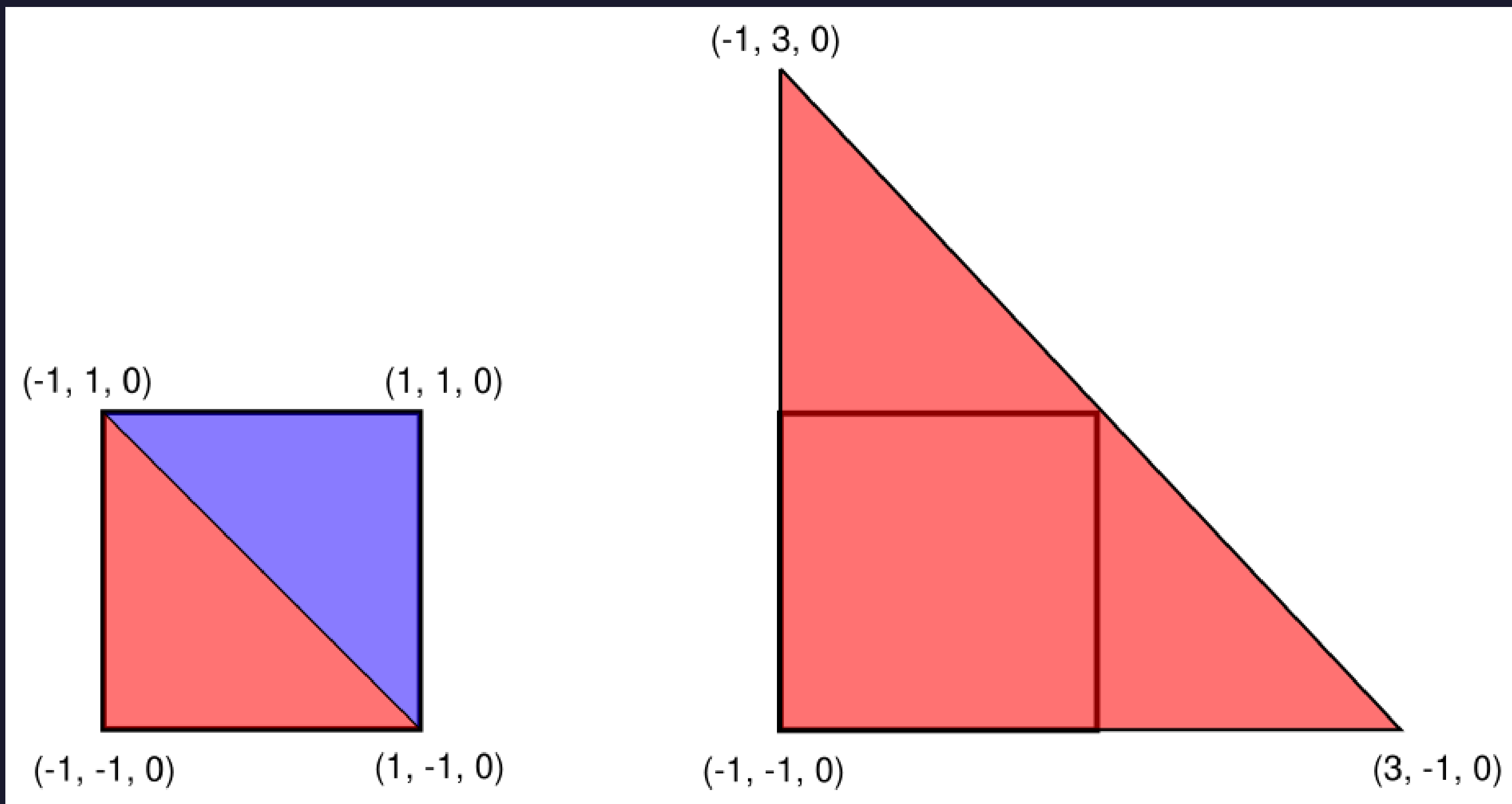
# Geometria suport (I)

De obicei, în realizarea unui efect de postprocesare, se utilizează procesări **la nivel de pixel** cu ajutorul programelor de tip *fragment shader*.

Astfel, se utilizează banda grafică de rasterizare prin desenarea unei **geometrii suport, care cuprinde tot ecranul**:

- Triunghi care acoperă tot ecranul;
- Patrulater care acoperă tot ecranul, ce este construit din 2 triunghiuri.

# Geometria suport (II)







## Geometria suport (III)

La desenarea geometriei suport, se oferă acces la eşantionarea obiectelor de tip textură în care se stochează informațiile geometrice și culoarea obținută după calculele de iluminare.

Rezultatul efectului de postprocesare este salvat în obiectul de tip textură dintr-un framebuffer diferit de cele utilizate.

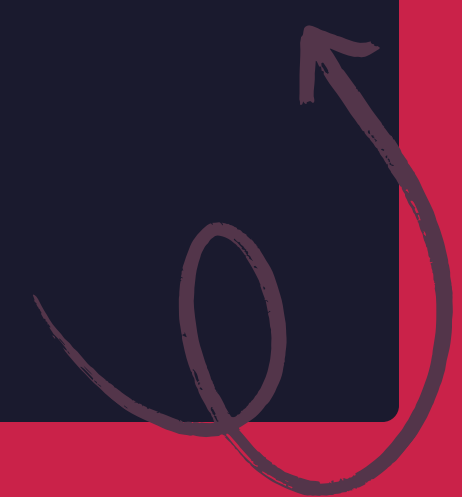
## Geometria suport (IV)

Exemplu de cod în limbajul *GLSL* pentru un program de tip *fragment shader*, care stabilește culoarea pixelului de la ieșire din obiectul de tip textură primit.

```
uniform sampler2D colorTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);
    out_color = texelFetch(colorTexture, texelCoord, 0);
}
```





# Geometria suport (V)

În situația în care rezoluția imaginii stocate în obiectul de tip textură primit la intrare este aceeași cu rezoluția obiectului de tip textură de culoare din obiectul de tip framebuffer în care se desenează, ce generează următorul cod?

```
uniform sampler2D colorTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);
    out_color = texelFetch(colorTexture, texelCoord, 0);
}
```



# Geometria suport (VI)

Ce generează următoarea secvență de cod?

```
uniform sampler2D colorTexture;
uniform sampler2D geometryPositionTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);

    vec3 viewSpacePosition =
        texelFetch(geometryPositionTexture, texelCoord, 0).xyz;

    if (viewSpacePosition.z > 1) {
        out_color = texelFetch(colorTexture, texelCoord, 0);
    }

    if (viewSpacePosition.z <= 1) {
        out_color = vec4(0);
    }
}
```







## Efectele de postprocesare în spațiul de culoare

Cele mai uzuale efecte de postprocesare sunt:

- Transformarea culorilor din spațiul RGB în nuanțe de gri - „*grayscale*”. În limba română, acest proces este cunoscut sub numele de trecere în alb-negru.
- Cartografierea tonurilor de culoare - „*Tone Mapping*”;
  - Utilizarea unei plaje dinamice de intensități pentru iluminare - „*High Dynamic Range*”;
- Hărți de culori - „*Tone Mapping Lookup Table*”



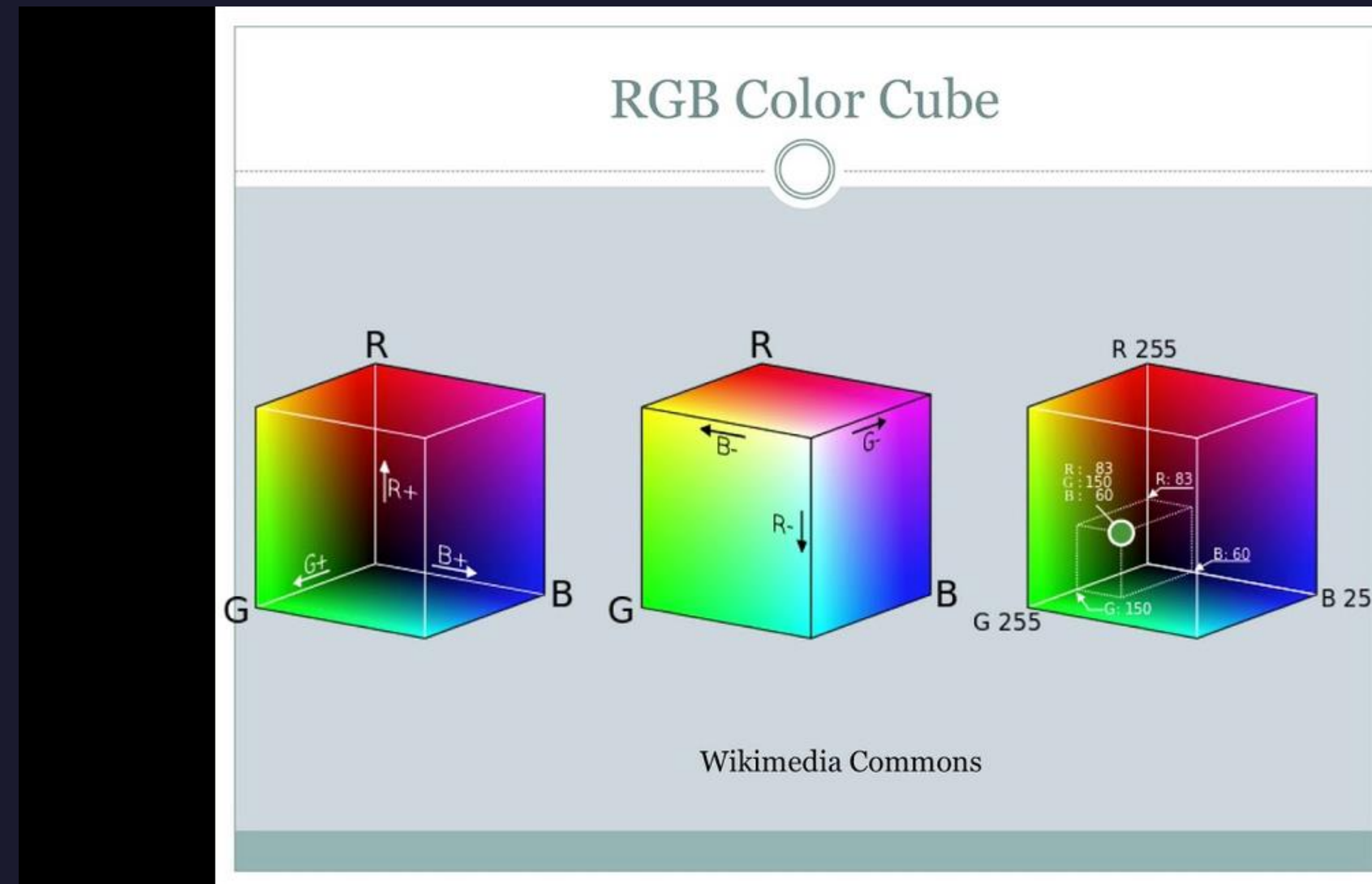
# Transformarea culorilor în nuanțe de gri (I)





# Transformarea culorilor în nuanțe de gri (II)

În modelul de culoare RGB, culorile care au **aceeași valoare** pe **toate cele 3 canale de culoare** reprezintă **nuanțe de gri**.



# Transformarea culorilor în nuanțe de gri (III)

În modelul de culoare RGB, culorile care au **aceeași valoare** pe **toate cele 3 canale de culoare** reprezintă **nuanțe de gri**.

```
uniform sampler2D colorTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);

    vec4 color = texelFetch(colorTexture, texelCoord, 0);

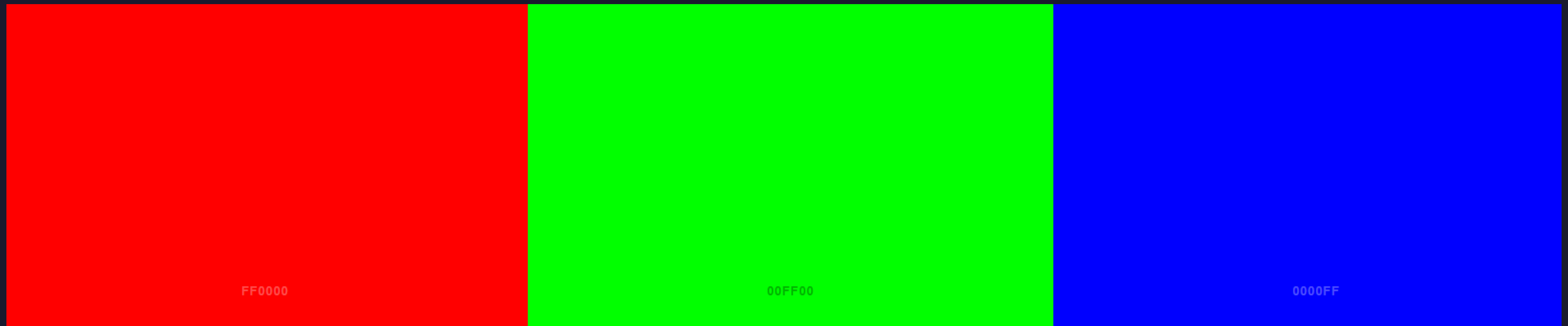
    float grayscale = (color.r + color.g + color.b) / 3.0f;

    out_color.rgb = vec3(grayscale);
}
```





# Transformarea culorilor în nuanțe de gri (IV)



# Transformarea culorilor în nuanțe de gri (V)

```
uniform sampler2D colorTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);

    vec4 color = texelFetch(colorTexture, texelCoord, 0);

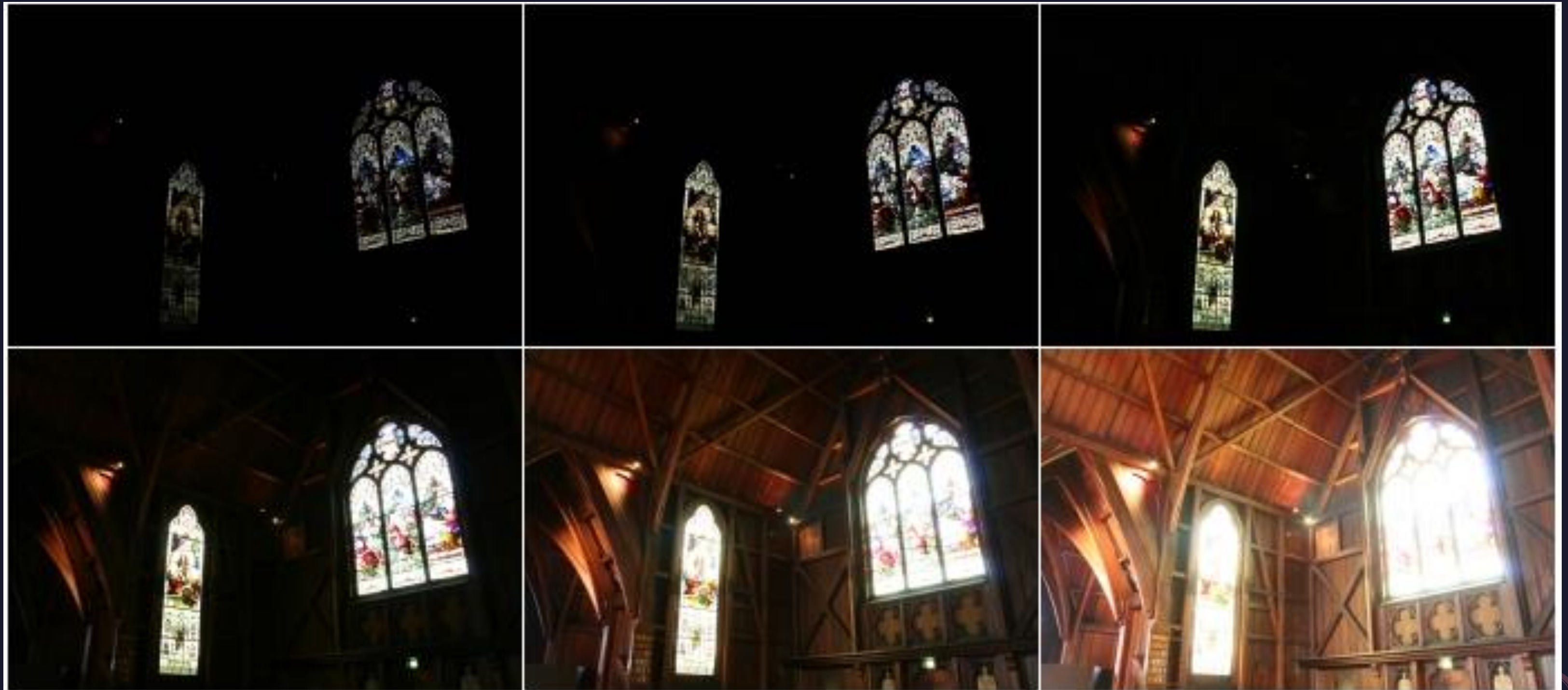
    float grayscale =
        0.21f * color.r +
        0.72f * color.g +
        0.07f * color.b;

    out_color.rgb = vec3 (grayscale);
}
```

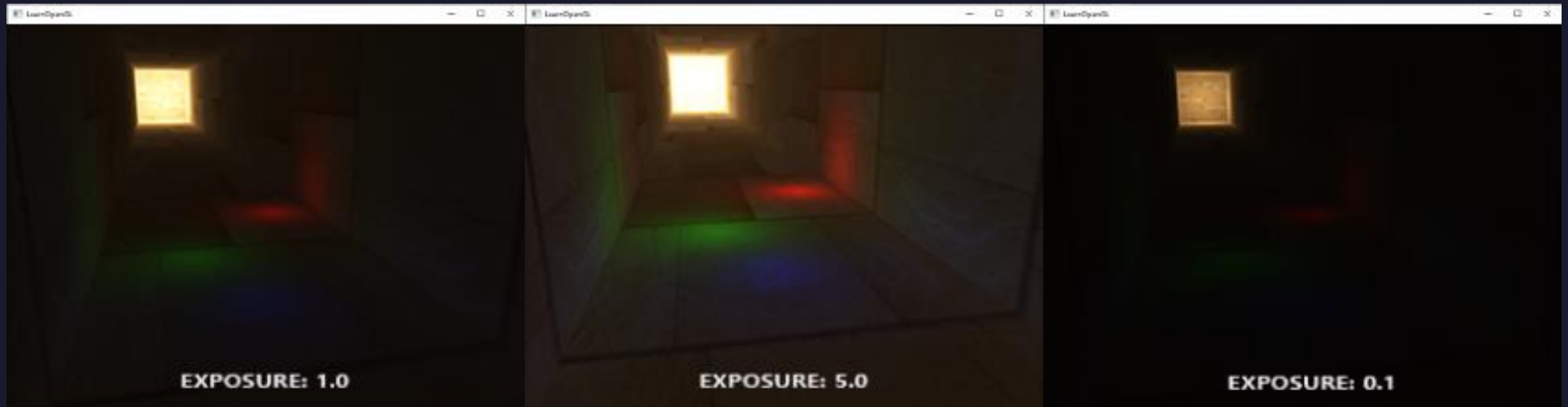




# Efectul de supra-iluminare (I)



# Efectul de supra-iluminare (II)





## Efectul de supra-iluminare (III)

În imaginile generate de calculator, la realizarea calculelor de iluminare, un fragment poate obține o **intensitate de iluminare ce depășește valoarea 1**.

Cu toate acestea, standardul de afișare a dispozitivelor de afișaj nu permit valori în afara intervalului  $[0, 1]$  pentru canalele de culoare din modelul de culoare RGB. Astfel, valorile canalelor de culoare trebuie aduse în intervalul  $[0, 1]$  după realizarea calculelor de iluminare.

Există mai multe standarde pentru acest proces, cunoscut sub numele de cartografiere a tonurilor de culoare - „***Tone Mapping***”. Nu există un singur standard general acceptat.



# Efectul de supra-iluminare (IV)

$$channel = 1 - e^{-channel \cdot exposure}$$

```
uniform float exposure;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);

    vec4 color = texelFetch(colorTexture, texelCoord, 0);

    color.rgb = vec3(1.0f) - exp(-color * exposure);

    out_color = color;
}
```







# Bibliografie

1. HDR (no date) LearnOpenGL. Available at: <https://learnopengl.com/Advanced-Lighting/HDR> (Accessed: 01 April 2026).
2. Hable, J. (2017) Filmic tonemapping with Piecewise Power curves, Filmic Worlds. Available at: <http://filmicworlds.com/blog/filmic-tonemapping-with-piecewise-power-curves/> (Accessed: 01 April 2026).





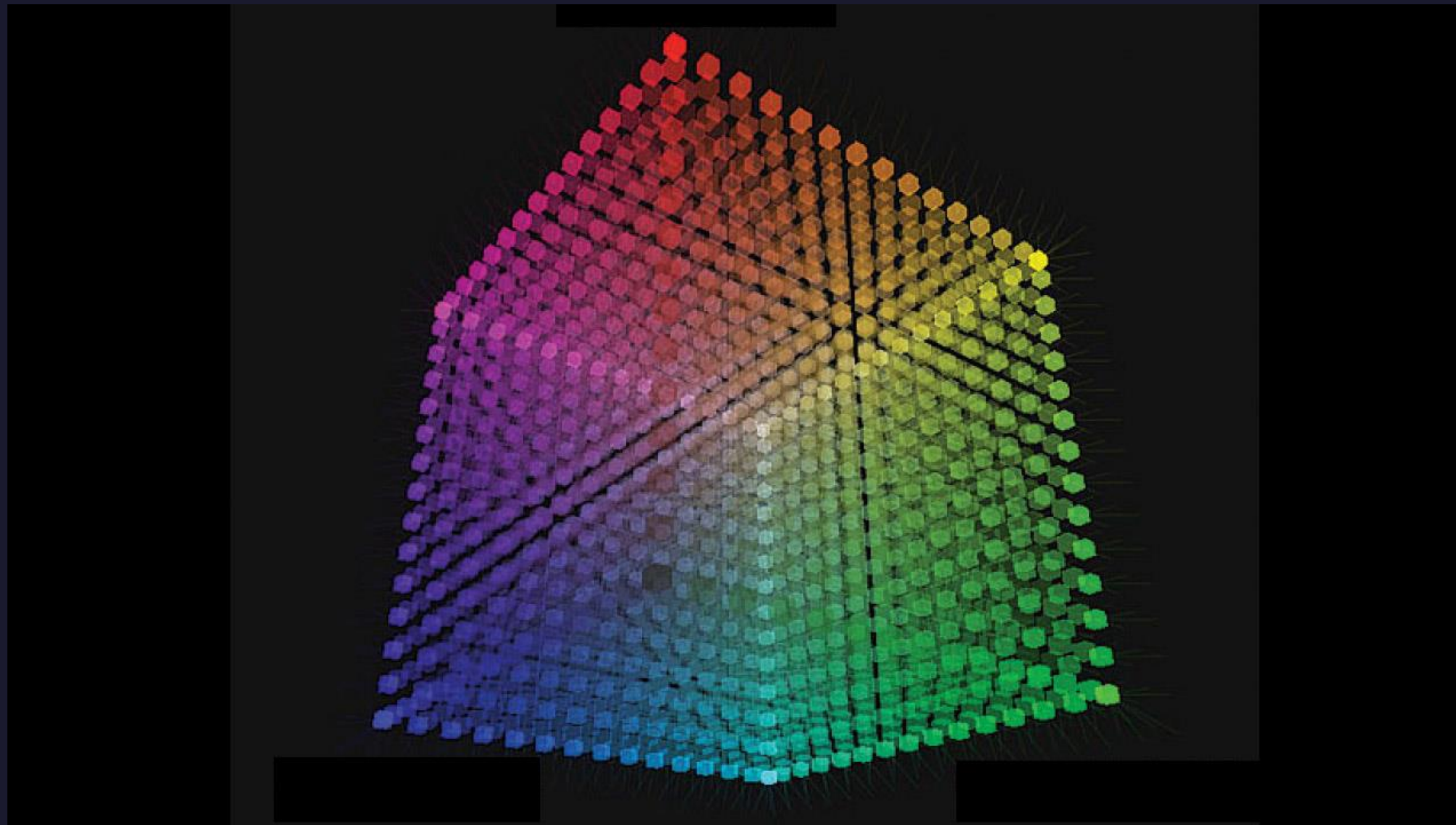
## Hărți de culori (I)

Pentru a oferi un control asupra culorilor afișate, motoarele grafice obișnuiesc să ofere artiștilor diferite unelte pentru ajustarea culorilor.

Una dintre cele mai utilizate abordări de modificare a culorilor este cea care oferă posibilitatea de a păstra o cartografiere a tonurilor de culoare pe baza unei tabele de culori 3D - „*Tone Mapping Lookup Table*”.



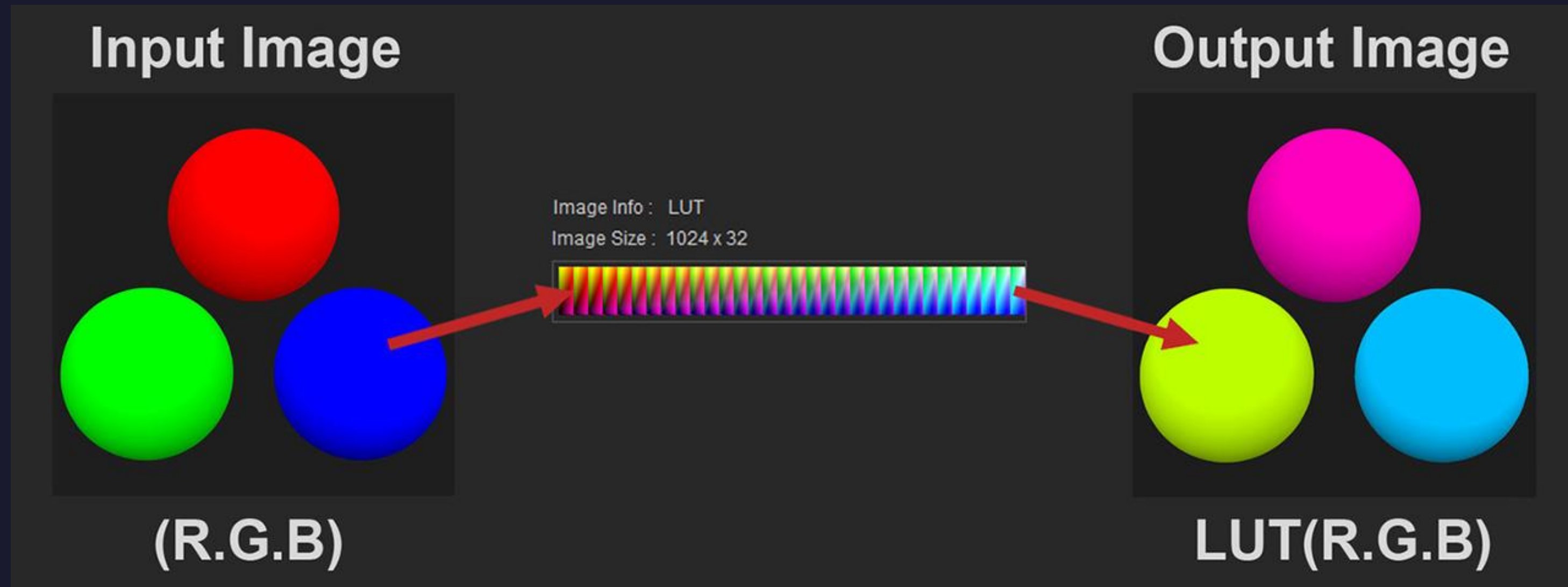
# Hărți de culori (II)





# Hărți de culori (III)

Se utilizează un obiect de tip textură 3D, care se eșantionează la coordonatele (r, g, b) ale culorii utilizate la intrare.



# Hărți de culori (IV)

Se utilizează un obiect de tip textură 3D, care se eşantionează la coordonatele (r, g, b) ale culorii utilizate la intrare.

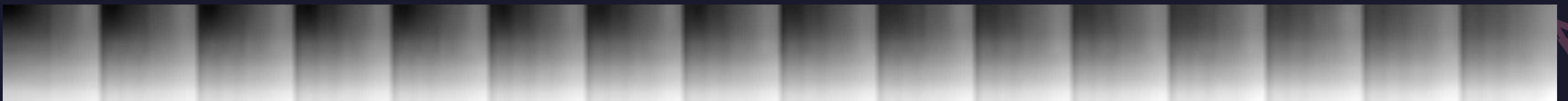
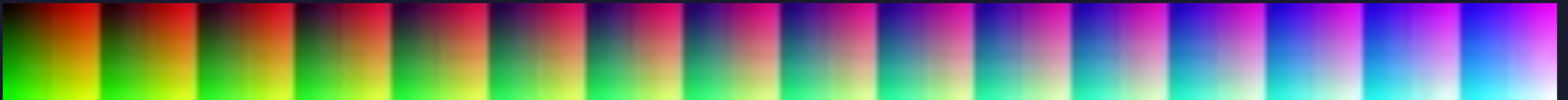
```
uniform sampler2D colorTexture;  
uniform sampler3D lutTexture;  
  
out vec4 out_color;  
  
void main()  
{  
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);  
  
    vec4 color = texelFetch(colorTexture, texelCoord, 0);  
  
    vec4 newColor = texture(lutTexture, color.rgb);  
  
    out_color = newColor;  
}
```





# Hărți de cartografiere a tonurilor de culoare (IV)

Avantajul utilizării unei astfel de abordări este faptul că imaginea care conține tabela de culori este 2D și poate fi modificată de către un artist într-o aplicație dedicată editării graficii raster, care, de obicei, are mult mai multe capacități de editare față de un motor grafic.



# Bibliografie

1. Selan, Jeremy. "Using lookup tables to accelerate color transformations." GPU Gems 2 (2005): 381-392.







## Procesul de netezire a imaginilor - „*blur*” (I)

Este un proces de netezire a valorilor din canalele de culoare a unui pixel pe baza culorilor pe care le are pixelii vecini.

Cea mai simplă abordare de obținere a unui efect de netezire este prin modificarea culorilor tuturor pixelilor pe baza unei medii ponderate a culorilor vecinilor pixelului.



## Procesul de netezire a imaginilor - „*blur*” (II)

```
uniform sampler2D colorTexture;

out vec4 out_color;

void main()
{
    ivec2 texelCoord = ivec2(gl_FragCoord.xy);

    vec4 sum = vec4(0.0);

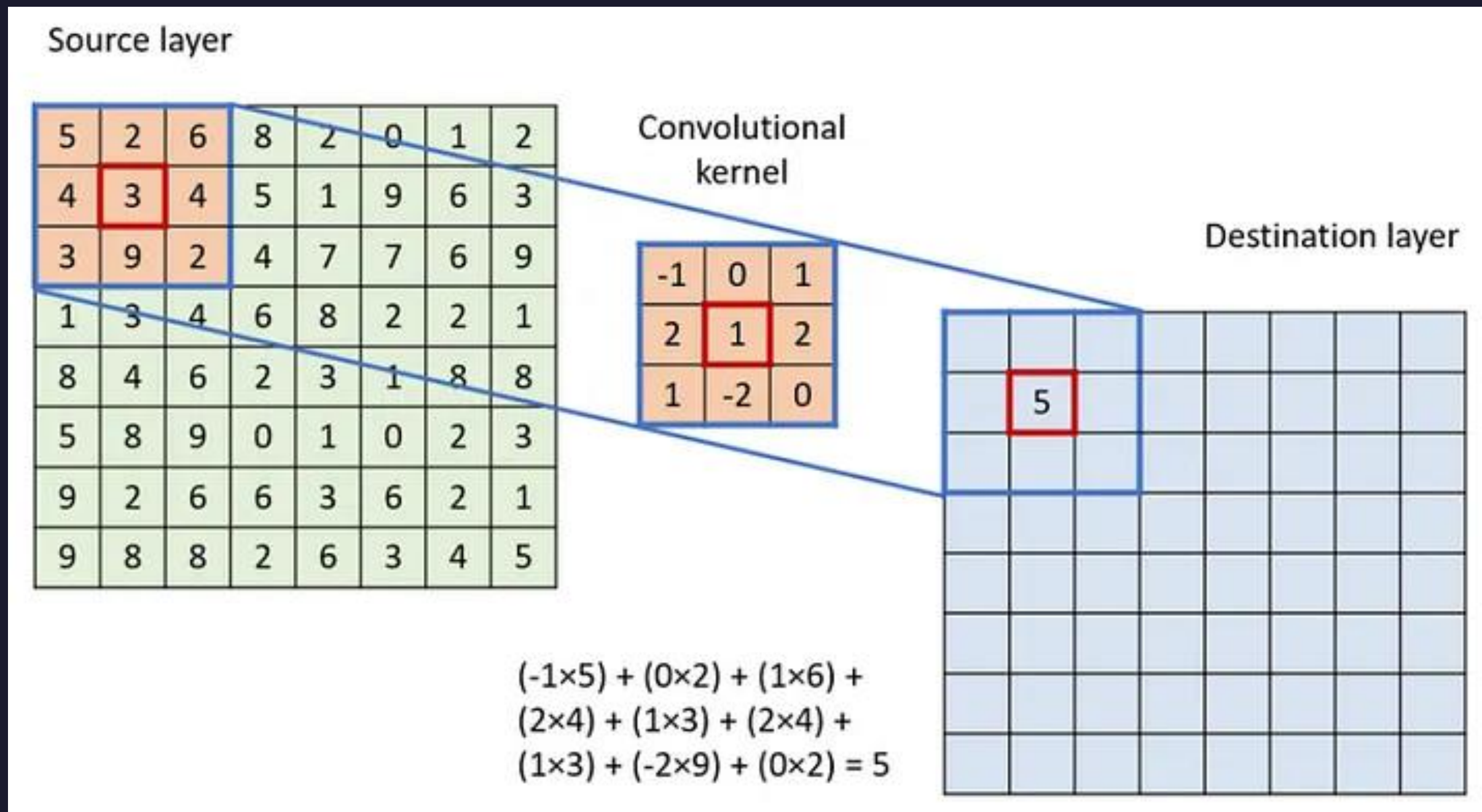
    for(int y = -1; y <= 1; y++) {
        for(int x = -1; x <= 1; x++) {
            sum += texelFetch(colorTexture, texelCoord + ivec2(x, y), 0);
        }
    }

    out_color = sum / 9.0;
}
```



# Procesul de netezire a imaginilor - „*blur*” (III)

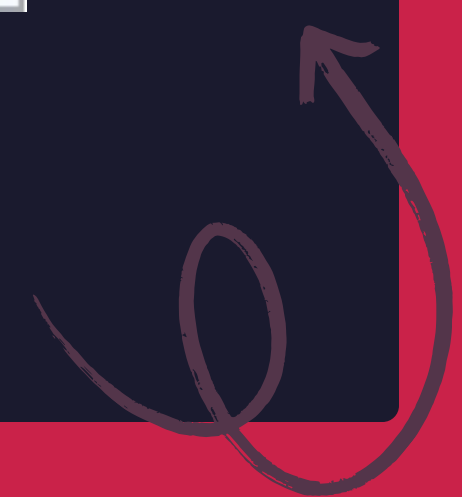
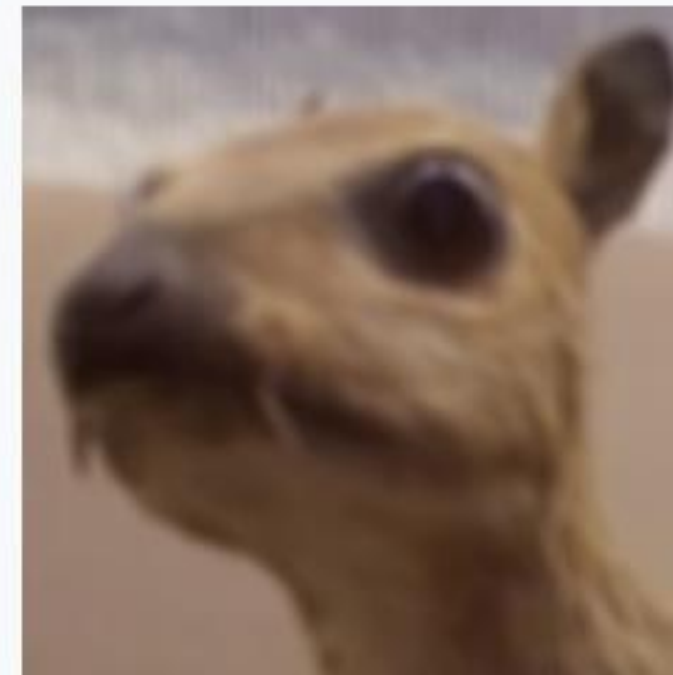
## Procesul de *convoluție*



# Procesul de netezire a imaginilor - „*blur*” (IV)

Procesul de *convoluție*

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

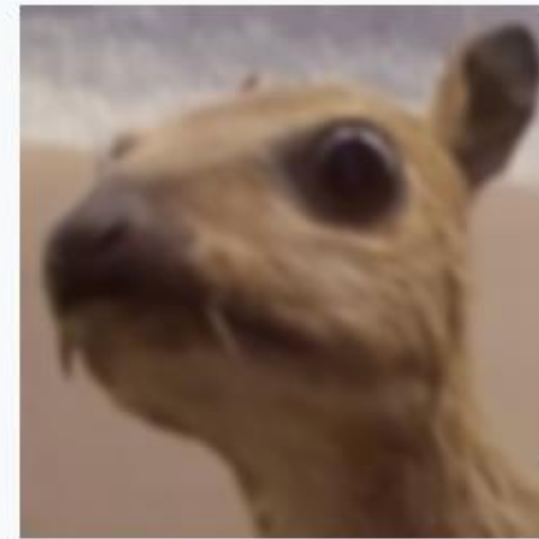




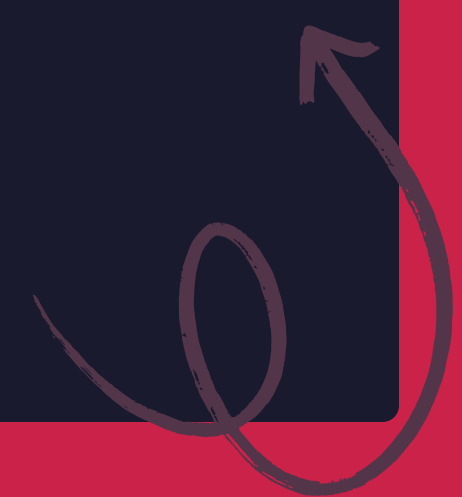
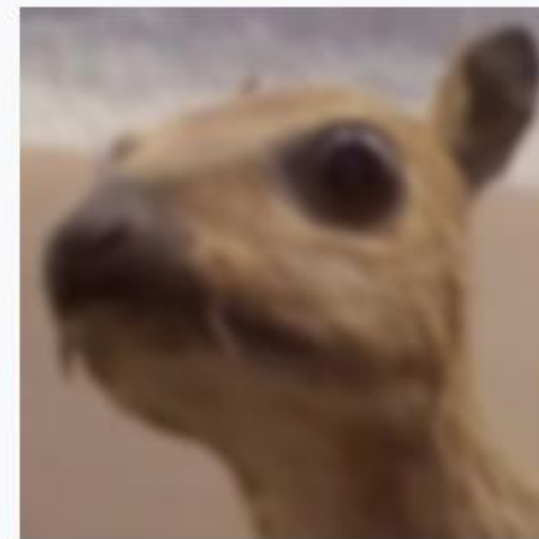
# Procesul de netezire a imaginilor - „*blur*” (V)

## Procesul de *convoluție*

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



$$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

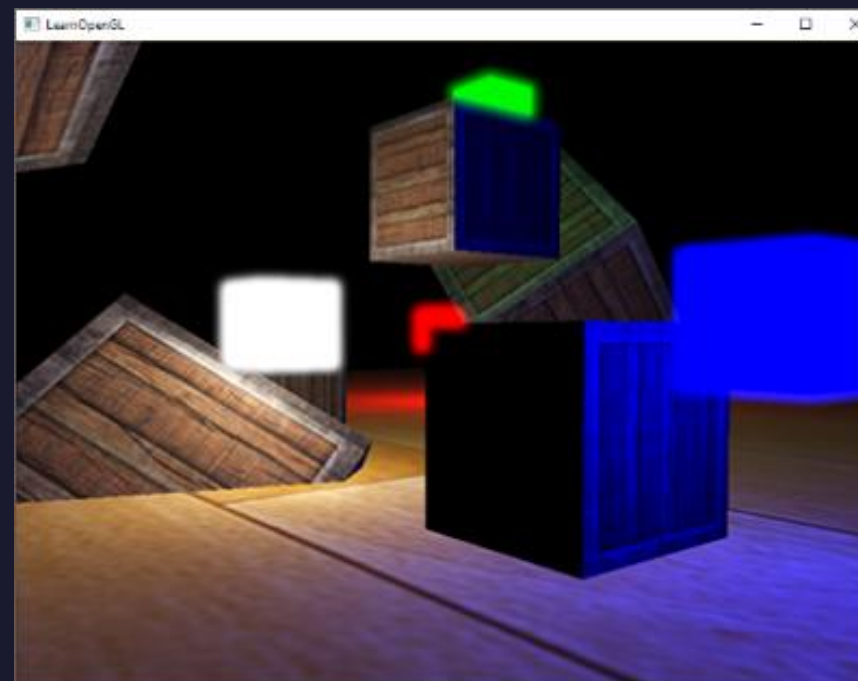
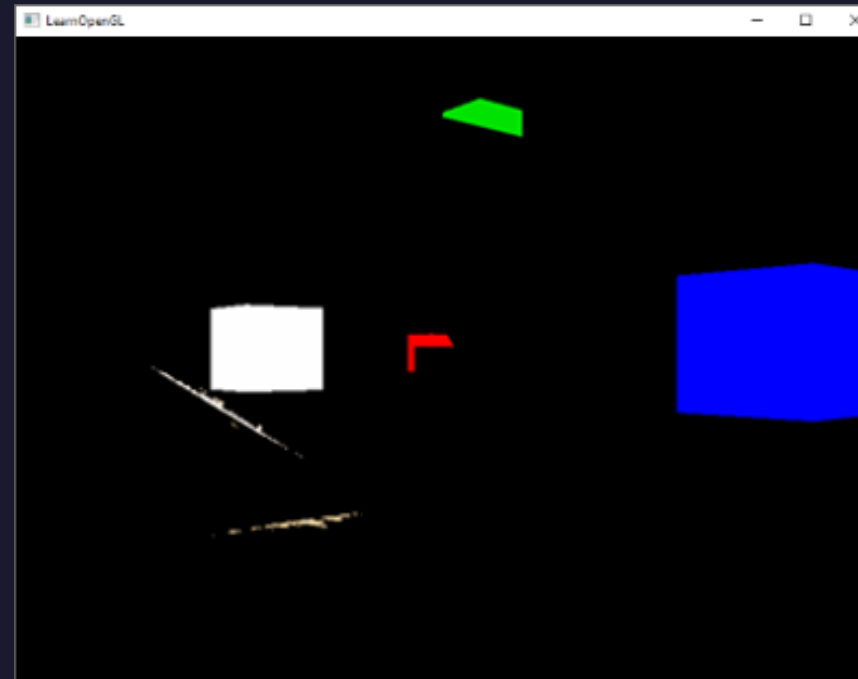
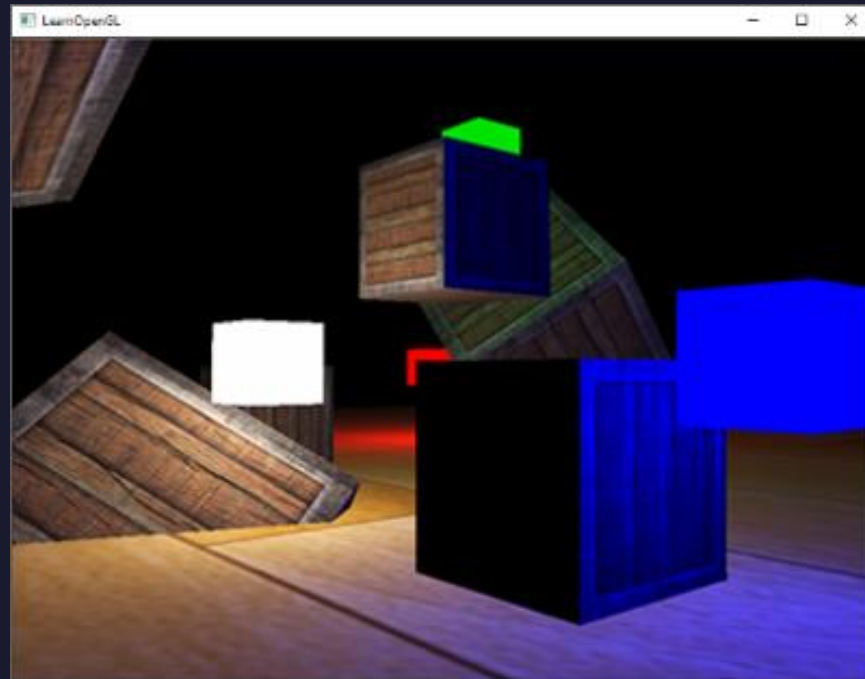


# Efectul de tip „*bloom*” (I)

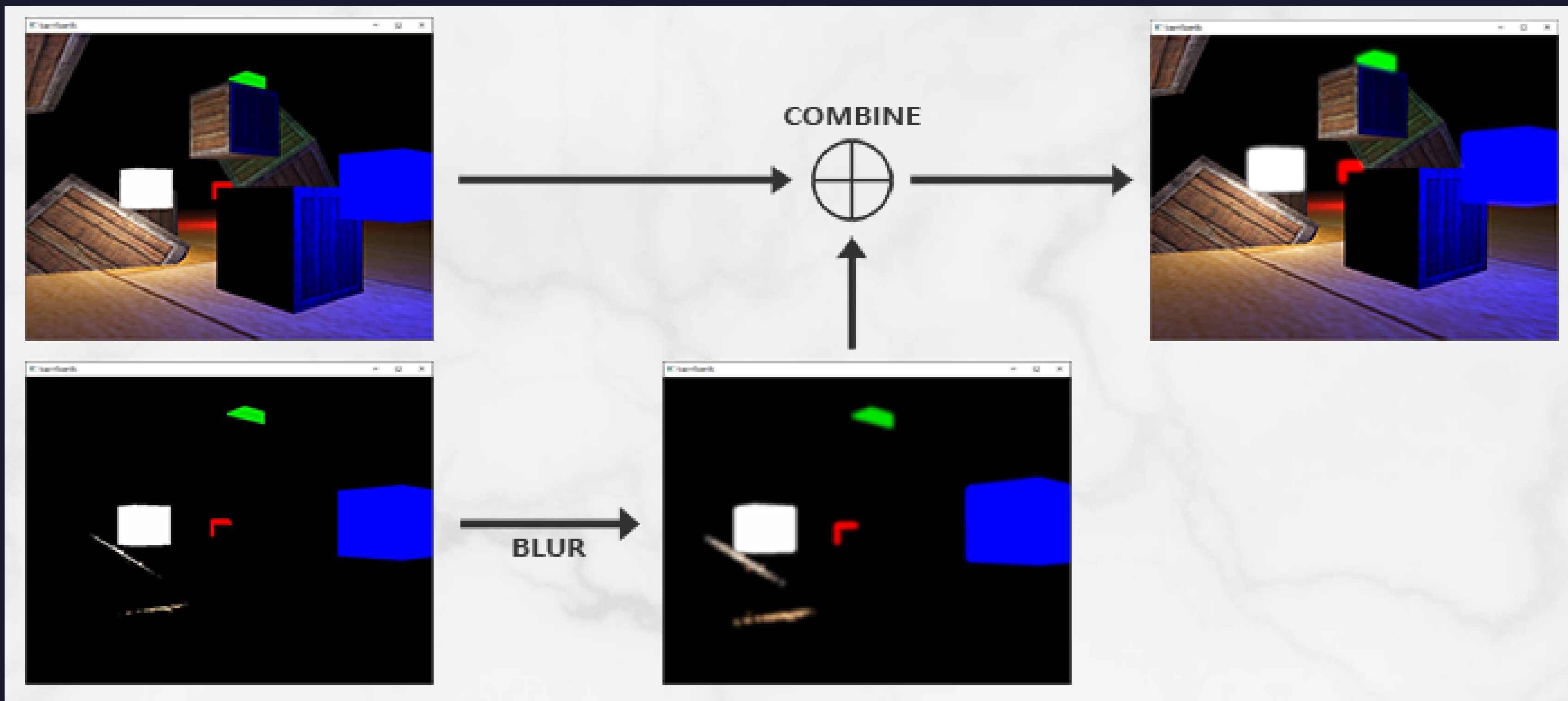




# Efectul de tip „*bloom*” (II)



# Efectul de tip „*bloom*” (III)





# Bibliografie

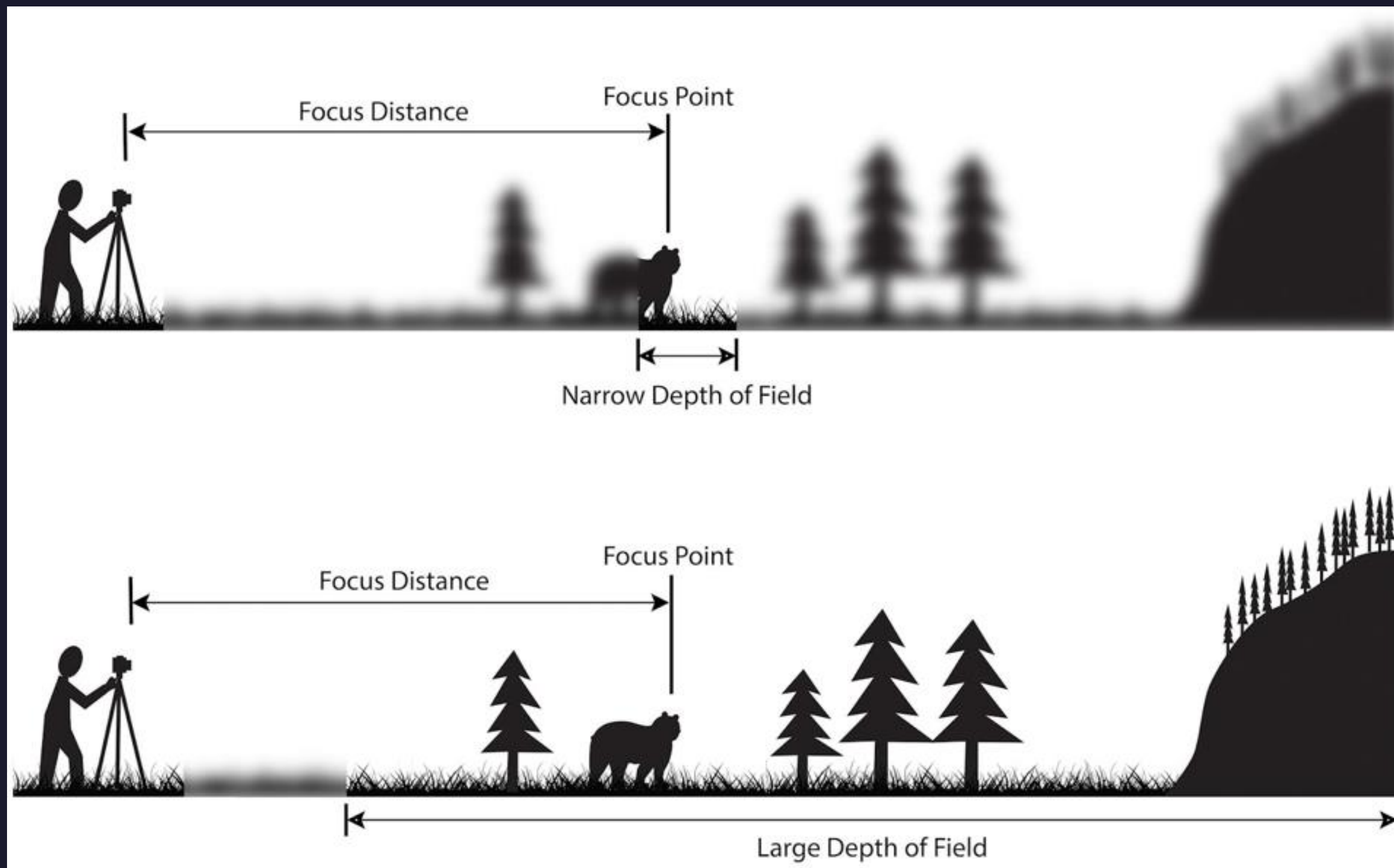
1. Bloom (no date) LearnOpenGL.  
Available at:  
<https://learnopengl.com/Advanced-Lighting/Bloom> (Accessed: 01 April 2026).

# Efectul de adâncime a câmpului vizual (I)





# Efectul de adâncime a câmpului vizual (II)





## Efectul de adâncime a câmpului vizual (III)

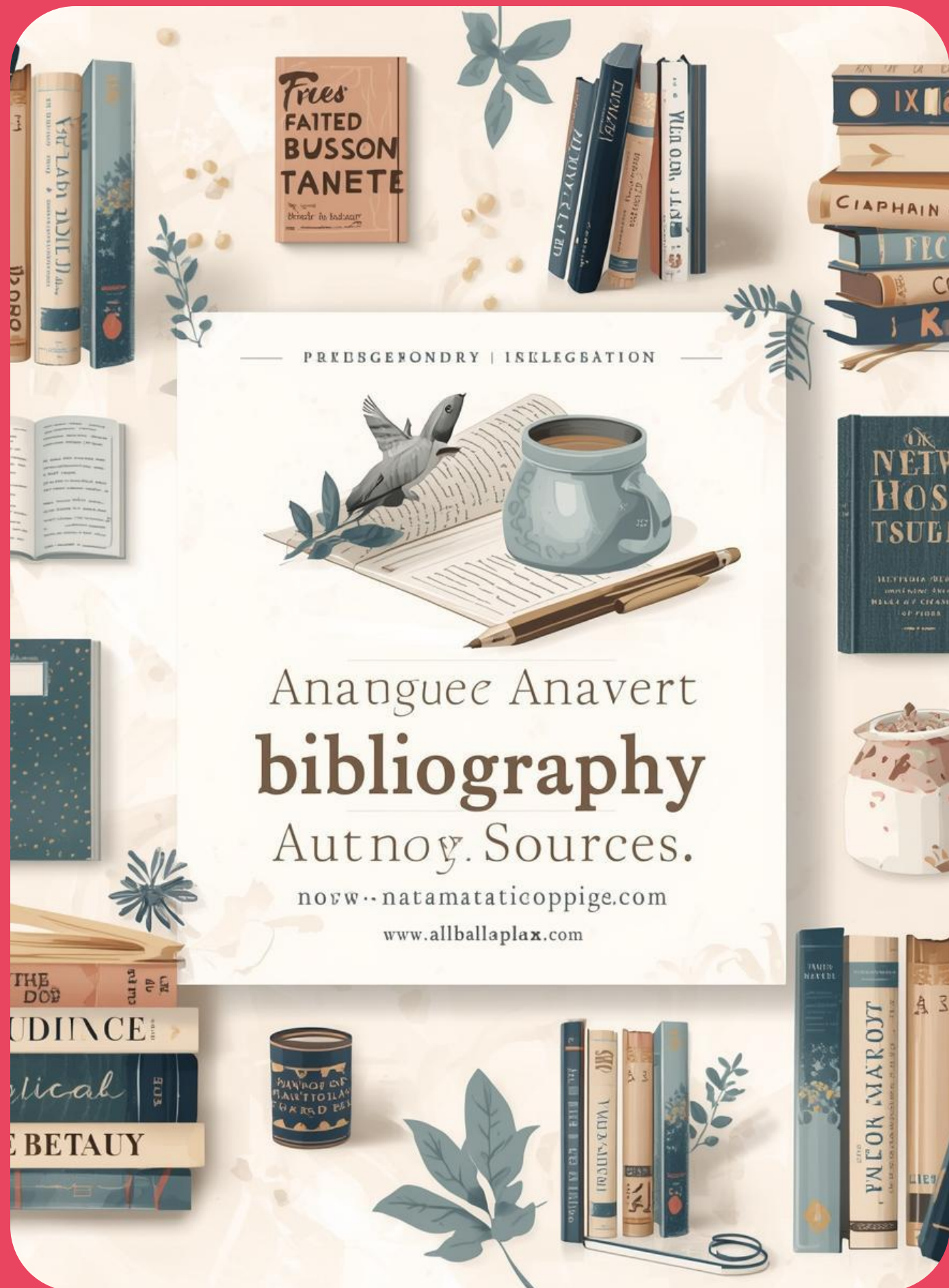
Se utilizeaza informatia geometrica de pozitie a fragmentului in spatiul de vizualizare ca sa se calculeze distanta de la fragment la observator. In situatia in care distanta nu se afla in intervalul de focus, pentru culoarea pixelului in care se deseneaza fragmentul se aplica un efect de netezire.

Pentru un efect interesant, coeficientul de netezire poate sa fie proportional cu distanta.



# Bibliografie

1. Akenine-Moller, Tomas, Eric Haines, and Naty Hoffman. Real-time rendering. AK Peters/crc Press, 2019.





# Efectul de iluminare volumetrică (I)







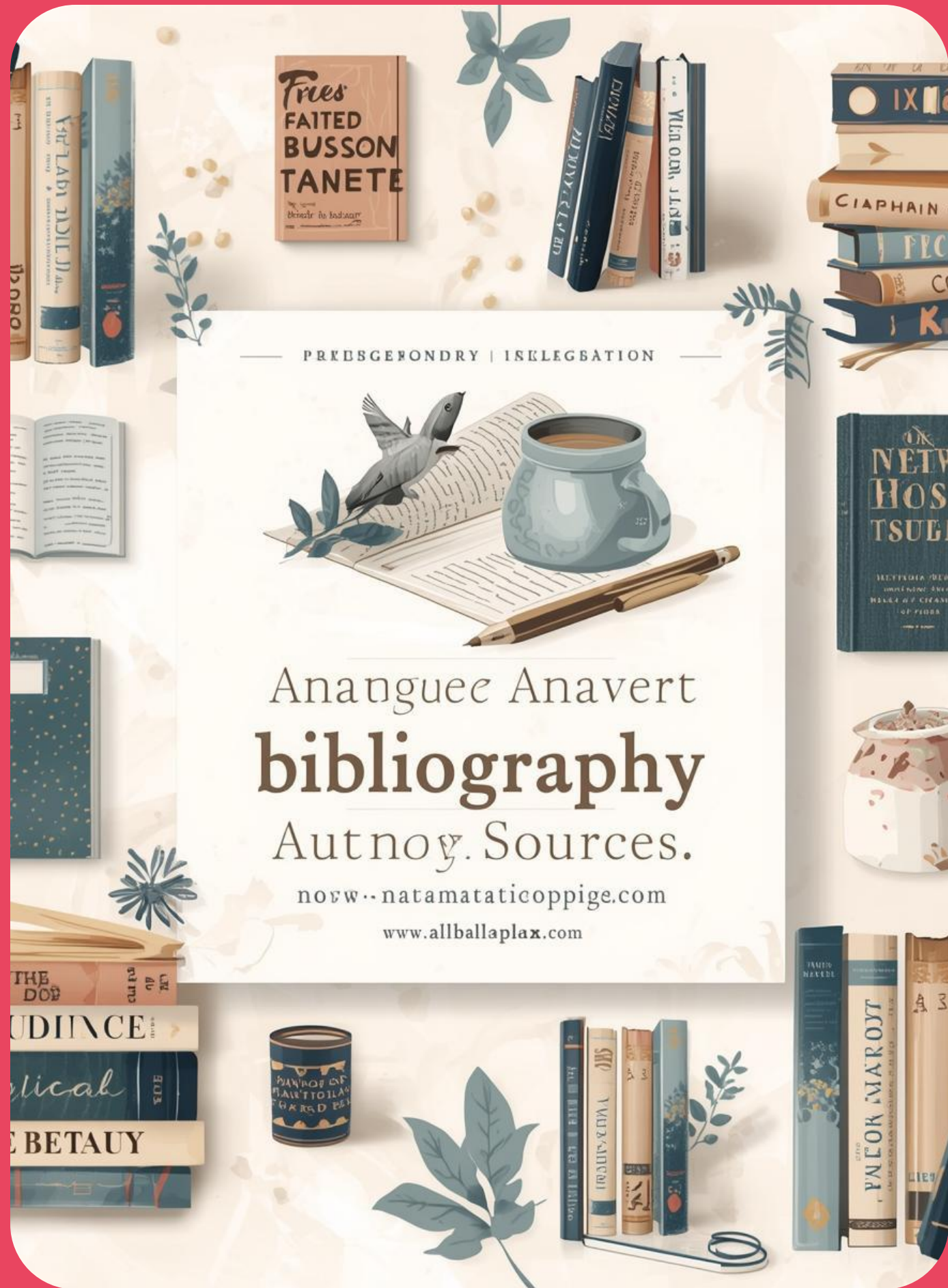
## Efectul de iluminare volumetrică (II)

Se desenează sursa de lumină în ecran, sub forma unui cerc de intensitate luminoasă mare.

Se face o parcurgere în spațiul ecranului de la coordonata pixelului în spațiul ecranului, de-a lungul direcției spre sursa de lumină.

# Bibliografie

1. Mitchell, Kenny. "Volumetric light scattering as a post-process." GPU Gems 3 (2007): 275-285.







# Recomandări de proiecte (I)

Echipe de o persoană:

- Cartografierea tonurilor de culoare - „*Tone Mapping*” prin utilizarea unei plaje dinamice de intensități pentru iluminare - „*High Dynamic Range*”;
- Hărți de culori - „*Tone Mapping Lookup Table*”;
- Efectul de tip „*bloom*”;
- Alte efecte de postprocesare, de complexitate mică, alese de studenți.





## Recomandări de proiecte (II)

Echipe de 2 persoane:

- Efectul de adâncime a câmpului vizual  
- „*depth of field*”;
- Efectul de iluminare volumetrică;
- Alte efecte de postprocesare, de complexitate medie, alese de studenți.





# Recomandări de proiecte (III)

Echipe de 3 persoane:

- Combinație de mai multe efecte de postprocesare;
- Alte efecte de postprocesare, de complexitate mare, alese de studenți.